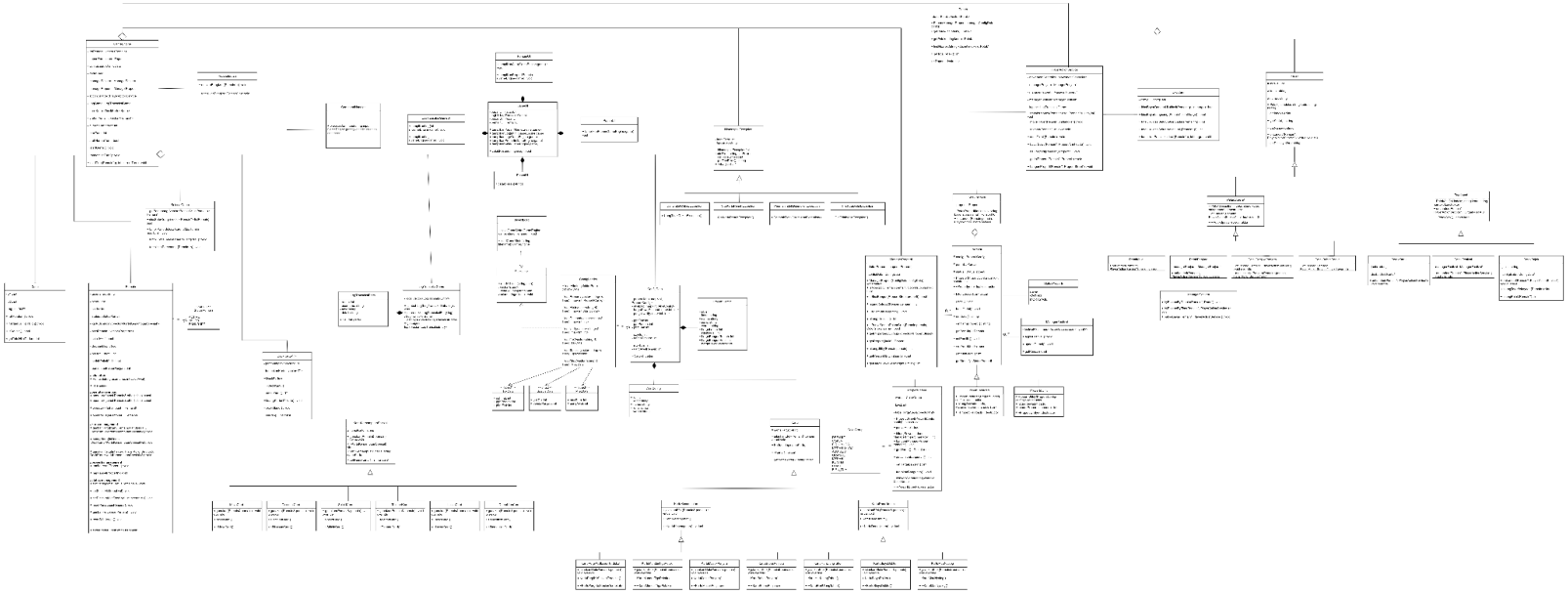
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : GTW
Nama Kelompok : nantidiisi
1. 13524042 / Suryani Mulia Utami
2. 13524073 / Keisha Rizka Syofyani
3. 13524087 / Muhammad Fakhry Zaki
4. 13524091 / Vara Azzara Ramli Pulukadang
5. 13524109 / Helena Kristela Sarhawa

GTW - nantidiisi

1. Diagram Kelas



Gambar 1.1 Diagram Kelas

Gambar diagram dapat diakses pada [Tubes 1 OOP.drawio]

2. Deskripsi Kelas

2.1. Kelas DeckKartu<T>

Kelas DeckKartu<T> merupakan kelas generic (template) yang memiliki tanggung jawab untuk mengelola tumpukan kartu dalam permainan. Kelas ini berperan dalam mekanisme pengambilan kartu, pembuangan kartu, serta pengacakan kartu. Kelas ini menyimpan

dua buah struktur data bertipe vector<T*> yaitu tumpukanKartu sebagai sumber kartu yang dapat diambil oleh pemain, serta pembuangan sebagai tempat pengumpulan kartu yang telah digunakan.

Metode ambilKartu() digunakan untuk mengambil kartu dari tumpukan utama dengan mengambil elemen paling belakang (back) dan melakukan pop_back. Apabila tumpukan utama telah habis, metode acakUlang() akan dipanggil secara otomatis di dalam ambilKartu() untuk memindahkan seluruh kartu dari pembuangan ke tumpukan utama lalu mengacaknya menggunakan std::shuffle dengan generator std::mt19937. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- vector<T*> tumpukanKartu — sumber kartu yang dapat diambil
- vector<T*> pembuangan — kartu yang telah digunakan
- DeckKartu() : sebagai konstruktor default
- ~DeckKartu() : sebagai destruktor
- void tambahKeKartu(T* kartu) : menambahkan kartu ke tumpukanKartu
- T* ambilKartu() : mengambil kartu dari tumpukan (mengacak ulang jika kosong, melempar runtime_error jika deck benar-benar kosong)
- void buangKartu(T* kartu) : memindahkan kartu ke tumpukan pembuangan
- void acakUlang() : memindahkan semua kartu dari pembuangan ke tumpukan lalu mengacaknya
- int ukuran() const : mengembalikan ukuran tumpukanKartu
- int ukuranPembuangan() const : mengembalikan ukuran tumpukan pembuangan
- bool kosong() const : mengembalikan true jika tumpukanKartu dan pembuangan sama-sama kosong
- const vector<T*>& getTumpukan() const : getter tumpukanKartu
- const vector<T*>& getPembuangan() const : getter pembuangan

2.2. Kelas Kartu

Kelas Kartu merupakan kelas abstrak yang menjadi dasar bagi semua kartu yang langsung dieksekusi saat diambil oleh pemain dari petak kartu. Kelas ini memiliki atribut namaKartu yang menyimpan nama kartu. Kelas ini mendefinisikan metode abstrak jalankanEfek(Pemain& pemain, PlayerActionService& svc) yang harus diimplementasikan oleh setiap kelas turunannya. Parameter PlayerActionService diteruskan agar efek kartu dapat memanggil aksi-aksi atomik seperti perpindahan pemain, transfer uang, dan

interaksi penjara. Kelas Kartu menjadi parent dari KartuKesempatan dan KartuDanaUmum. Setelah dijalankan, kartu dikembalikan ke DeckKartu via buangKartu(). Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- string namaKartu — nama kartu yang disimpan
- explicit Kartu(const string& nama) : konstruktor
- virtual ~Kartu() : destruktor virtual
- virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0 : pure virtual function
- const string& getNamaKartu() const : getter nama kartu

2.3. Kelas KartuKesempatan

Kelas KartuKesempatan merupakan kelas abstrak turunan dari Kartu yang menjadi basis bagi kartu-kartu kesempatan. Kelas ini memiliki empat kelas turunan konkret: KartuPergiKeStasiunTerdekat, KartuMundurTigaPetak, KartuMasukPenjara, dan KartuBebasPenjara. Setiap kelas turunan mengimplementasikan jalankanEfek() dengan mendelegasikan aksi ke PlayerActionService, misalnya moveToNearestStation(), movePlayerRelative(-3), sendToJail(), atau escapeFromJailByCard(). Sesuai spesifikasi, deck kesempatan berisi keempat kartu ini dan dikocok via isiDeckKesempatan(). Berikut daftar atribut dan metode.

- explicit KartuKesempatan(const string& nama) : konstruktor meneruskan ke Kartu
- virtual ~KartuKesempatan() : destruktor virtual
- virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0 : pure virtual

2.4. Kelas KartuPergiKeStasiunTerdekat

Kelas KartuPergiKeStasiunTerdekat merupakan kelas turunan dari KartuKesempatan dengan nama kartu "Pergi ke stasiun terdekat." Melalui jalankanEfek(), kartu ini memanggil svc.moveToNearestStation(pemain) untuk memindahkan pemain ke stasiun yang paling dekat dari posisinya saat ini. Berikut daftar atribut dan metode yang digunakan.

- KartuPergiKeStasiunTerdekat() : konstruktor, meneruskan nama kartu ke KartuKesempatan

- ~KartuPergiKeStasiunTerdekat() = default
- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override

2.5. Kelas KartuMundurTigaPetak

Kelas KartuMundurTigaPetak merupakan kelas turunan dari KartuKesempatan dengan nama kartu "Mundur 3 petak." Melalui jalankanEfek(), kartu ini memanggil svc.movePlayerRelative(pemain, -3) untuk memundurkan pemain sebanyak tiga petak. Nilai negatif pada movePlayerRelative menandakan pergerakan mundur. Berikut daftar atribut dan metode.

- KartuMundurTigaPetak() : konstruktor
- ~KartuMundurTigaPetak() = default
- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override

2.6. Kelas KartuMasukPenjara

Kelas KartuMasukPenjara merupakan kelas turunan dari KartuKesempatan dengan nama kartu "Masuk Penjara." Melalui jalankanEfek(), kartu ini memanggil svc.sendToJail(pemain) untuk memindahkan pemain ke petak penjara dengan status tahanan. Berikut daftar atribut dan metode.

- KartuMasukPenjara() : konstruktor
- ~KartuMasukPenjara() = default
- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override

2.7. Kelas KartuBebasPenjara

Kelas KartuBebasPenjara merupakan kelas turunan dari KartuKesempatan dengan nama kartu "Bebas dari Penjara." Melalui jalankanEfek(), kartu ini memanggil svc.escapeFromJailByCard(pemain) untuk membebaskan pemain dari status tahanan penjara menggunakan kartu. Berikut daftar atribut dan metode.

- KartuBebasPenjara() : konstruktor
- ~KartuBebasPenjara()
- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override

2.8. Kelas KartuDanaUmum

Kelas KartuDanaUmum merupakan kelas abstrak turunan dari Kartu yang menjadi basis bagi kartu-kartu dana umum. Kelas ini memiliki tiga kelas turunan konkret: KartuHariUlangTahun, KartuBiayaDokter, dan KartuMauNyaleg. Efek setiap kartu berkaitan dengan perubahan saldo pemain menggunakan mekanisme transfer melalui PlayerActionService. Sesuai spesifikasi, deck dana umum berisi tiga kartu ini dan dikocok via isiDeckDanaUmum(). Berikut daftar atribut dan metode.

- explicit KartuDanaUmum(const string& nama) : konstruktor meneruskan ke Kartu
- virtual ~KartuDanaUmum() : destruktor virtual
- virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0 : pure virtual

2.9. Kelas KartuHariUlangTahun

Kelas KartuHariUlangTahun merupakan kelas turunan dari KartuDanaUmum dengan nama kartu "Ini adalah hari ulang tahun Anda. Dapatkan M100 dari setiap pemain". Konstanta JUMLAH = 100 mendefinisikan jumlah yang diterima. Melalui jalankanEfek(), kartu ini memanggil svc.collectFromAllPlayers(pemain, JUMLAH) untuk mengumpulkan M100 dari setiap pemain aktif lainnya. Berikut daftar atribut dan metode.

- KartuHariUlangTahun() : konstruktor
- ~KartuHariUlangTahun()

- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override

2.10. Kelas KartuBiayaDokter

Kelas KartuBiayaDokter merupakan kelas turunan dari kelas KartuDanaUmum. Kelas ini melalui jalankanEfek() mengimplementasikan mekanisme pembayaran M700 dari pemilik kartu. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override
- KartuBiayaDokter()
- ~KartuBiayaDokter() = default

2.11. Kelas KartuMauNyaleg

Kelas KartuMauNyaleg merupakan kelas turunan dari kelas KartuDanaUmum. Kelas ini melalui jalankanEfek() mengimplementasikan mekanisme pembayaran M200 dari pemilik kartu kepada setiap pemain. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override
- KartuMauNyaleg()
- ~KartuMauNyaleg() = default

2.12. Kelas KartuKemampuanSpesial

Kelas KartuKemampuanSpesial merupakan kelas abstrak yang merepresentasikan kartu kemampuan spesial yang dapat disimpan oleh pemain (maksimal 3 di tangan) dan digunakan secara strategis sebelum melempar dadu. Berbeda dengan Kartu, KartuKemampuanSpesial tidak langsung dieksekusi saat didapatkan melainkan disimpan di tangan pemain. Kelas ini mendefinisikan pure virtual function gunakan(Pemain&, PlayerActionService&) dan getDeskripsi() yang diimplementasikan oleh setiap turunan. Terdapat pula getNilaiState() dan getSisaDurasiState() dengan implementasi default mengembalikan string kosong, digunakan untuk

keperluan save/load state kartu. Sesuai implementasi, deck skill diisi via isiDeckSkill() dengan kartu-kartu standar maupun kartu bonus. Berikut daftar atribut dan metode.

- string namaKartu — nama kartu
- explicit KartuKemampuanSpesial(const string& nama) : konstruktor
- virtual ~KartuKemampuanSpesial() : destruktor virtual
- virtual void gunakan(Pemain& pemain, PlayerActionService& svc) = 0 : pure virtual
- virtual string getDeskripsi() const = 0 : pure virtual, deskripsi efek kartu
- virtual string getNilaiState() const { return ""; } : untuk serialisasi nilai kartu (override di MoveCard, DiscountCard)
- virtual string getSisaDurasiState() const { return ""; } : untuk serialisasi sisa durasi (override di DiscountCard)
- const string& getNamaKartu() const : getter nama kartu

2.13. Kelas MoveCard

Kelas KartuKemampuanSpesial merupakan kelas abstrak yang merepresentasikan kartu kemampuan spesial yang dapat disimpan oleh pemain (maksimal 3 di tangan) dan digunakan secara strategis sebelum melempar dadu. Berbeda dengan Kartu, KartuKemampuanSpesial tidak langsung dieksekusi saat didapatkan melainkan disimpan di tangan pemain. Kelas ini mendefinisikan pure virtual function gunakan(Pemain&, PlayerActionService&) dan getDeskripsi() yang diimplementasikan oleh setiap turunan. Terdapat pula getNilaiState() dan getSisaDurasiState() dengan implementasi default mengembalikan string kosong, digunakan untuk keperluan save/load state kartu. Sesuai implementasi, deck skill diisi via isiDeckSkill() dengan kartu-kartu standar maupun kartu bonus. Berikut daftar atribut dan metode.

- string namaKartu — nama kartu
- explicit KartuKemampuanSpesial(const string& nama) : konstruktor
- virtual ~KartuKemampuanSpesial() : destruktor virtual
- virtual void gunakan(Pemain& pemain, PlayerActionService& svc) = 0 : pure virtual
- virtual string getDeskripsi() const = 0 : pure virtual, deskripsi efek kartu

- virtual string getNilaiState() const { return ""; } : untuk serialisasi nilai kartu (override di MoveCard, DiscountCard)
- virtual string getSisaDurasiState() const { return ""; } : untuk serialisasi sisa durasi (override di DiscountCard)
- const string& getNamaKartu() const : getter nama kartu

2.14. Kelas DiscountCard

Kelas DiscountCard merupakan kelas turunan dari KartuKemampuanSpesial. Atribut persenDiskon (int) ditentukan acak saat kartu dibuat (randRange(1,5)*10, yaitu 10%, 20%, 30%, 40%, atau 50%) dan sisaDurasi menyimpan sisa giliran efek berlaku. Masa berlaku DiscountCard hanya 1 giliran. DiscountCard berjumlah 3 lembar di dalam deck. Kartu ini mengimplementasikan getNilaiState() dan getSisaDurasiState() untuk mendukung fitur save/load. Berikut daftar atribut dan metode.

- int persenDiskon — persentase diskon, acak: 10/20/30/40/50
- int sisaDurasi — sisa durasi berlakunya efek
- explicit DiscountCard(int persen) : konstruktor
- ~DiscountCard()
- void gunakan(Pemain& pemain, PlayerActionService& svc) override
- string getDeskripsi() const override
- string getNilaiState() const override
- string getSisaDurasiState() const override
- int getPersenDiskon() const : getter persenDiskon
- int getSisaDurasi() const : getter sisaDurasi
- void setSisaDurasi(int d) : setter sisaDurasi

2.15. Kelas ShieldCard

Kelas ShieldCard merupakan kelas turunan dari kelas KartuKemampuanSpesial. Kelas ini melalui gunakan() mengimplementasikan mekanisme perlindungan pemain dari tagihan sewa maupun sanksi apapun yang merugikan. Masa berlaku ShieldCard hanyalah satu giliran. ShieldCard berjumlah 2 lembar secara keseluruhan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void gunakan(Pemain& pemain) override
- ShieldCard()
- ~ShieldCard()
- string getDeskripsi() const override

2.16. Kelas TeleportCard

Kelas TeleportCard merupakan kelas turunan dari kelas KartuKemampuanSpesial. Kelas ini melalui gunakan() mengimplementasikan mekanisme perpindahan pemain ke petak manapun di atas papan permainan. TeleportCard berjumlah 2 lembar secara keseluruhan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void gunakan(Pemain& pemain) override
- TeleportCard()
- ~TeleportCard()
- string getDeskripsi() const override

2.17. Kelas LassoCard

Kelas LassoCard merupakan kelas turunan dari kelas KartuKemampuanSpesial. Kelas ini melalui gunakan() mengimplementasikan mekanisme penarikan satu pemain lawan yang berada di depan posisi pemain saat ini ke petak tempat pemain tersebut berada. LassoCard berjumlah 2 lembar secara keseluruhan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void gunakan(Pemain& pemain) override
- LassoCard()
- ~LassoCard()
- string getDeskripsi() const override

2.18. Kelas DemolitionCard

Kelas DemolitionCard merupakan kelas turunan dari kelas KartuKemampuanSpesial. Kelas ini melalui gunakan() mengimplementasikan mekanisme penghancuran satu properti milik pemain lawan. DemolitionCard berjumlah 2 lembar secara keseluruhan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- void gunakan(Pemain& pemain) override
- DemolitionCard()
- ~DemolitionCard()
- string getDeskripsi() const override

2.19. Kelas Pemain

Kelas Pemain merupakan kelas yang merepresentasikan entitas pemain dalam permainan. Kelas ini menyimpan seluruh informasi yang berkaitan dengan kondisi pemain selama permainan berlangsung. Atribut yang dimiliki oleh kelas Pemain meliputi username sebagai identitas pemain, posisi pemain pada papan, jumlah uang pemain, status pemain (ACTIVE, JAILED, BANKRUPT) yang direpresentasikan menggunakan tipe enumerasi, daftar kartu kemampuan spesial yang dimiliki pemain di tangan, daftar properti yang dimiliki, status perlindungan pemain, status diskon pemain, besar diskon yang berlaku, penanda penggunaan skill dalam satu giliran, dan jumlah percobaan keluar dari penjara.

Metode dalam kelas ini mendukung berbagai aksi pemain seperti mengubah posisi pemain, menambahkan kartu ke tangan, membuang kartu, serta menggunakan kartu kemampuan spesial yang dimiliki. Kelas Pemain juga menyediakan operator overload untuk membandingkan pemain dan mengubah saldo, serta berbagai fungsi getter untuk mengakses atribut pemain. Selain itu, kelas ini

juga menyediakan fungsi untuk mengatur status pemain, mengelola penggunaan skill, serta mengatur percobaan keluar dari penjara. Kelas ini berinteraksi dengan kelas KartuKemampuanSpesial dalam hal penyimpanan dan penggunaan kartu, dan berinteraksi dengan kelas DeckKartu untuk memperoleh dan mengembalikan kartu. Pergerakan pemain dapat melibatkan komponen lain dalam sistem. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- string username
- int posisi
- int saldo
- StatusPemain status
- vector<KartuKemampuanSpesial*> kartuDiTangan
- vector<Properti*> asetPemain
- bool shieldAktif
- bool discountAktif
- int persenDiskon
- bool sudahPakaiSkill
- int percobaanKeluarPenjara
- Pemain(const string& username, int saldoAwal)
- ~Pemain()

Operator overload:

- Pemain& operator+=(int amount)
- Pemain& operator-=(int amount)
- bool operator>(const Pemain& other) const
- bool operator<(const Pemain& other) const

Manajemen posisi:

- void setPosisi(int pos)

Manajemen kartu:

- void tambahKartu(KartuKemampuanSpesial*, DeckKartu<KartuKemampuanSpesial>&)
- void buangKartu(int index, DeckKartu<KartuKemampuanSpesial>&)
- void gunakanKartu(int index, PlayerActionService&, DeckKartu<KartuKemampuanSpesial>&, bool)

Manajemen properti:

- void tambahAset(Properti*)
- void hapusAset(Properti*)
- Manajemen status:
- void setStatus(const StatusPemain&)
- void setShieldAktif(bool)
- void setDiscountAktif(bool, int persen = 0)
- void resetSkillUsage()
- void resetPercobaanPenjara()
- void tambahPercobaanPenjara()

- Seluruh getter dan setter yang diperlukan untuk mengakses data Pemain

2.20. Kelas NimonspoliException

Merupakan kelas utama yang merupakan turunan dari std::exception dan menjadi dasar bagi seluruh exception dalam sistem Nimonspoli. Kelas ini digunakan untuk merepresentasikan kondisi kesalahan yang dapat menyebabkan pelanggaran invariant dalam permainan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- int kodeError
- string tipeError
- int getkodeError()
- string gettipeError()
- virtual const char* what(): mengembalikan pesan error dalam bentuk const char*

2.21. Kelas UangTidakCukupException

Merupakan kelas turunan dari NimonspoliException untuk menangani kondisi ketika pemain tidak memiliki uang yang cukup untuk melakukan aksi yang memerlukan pengeluaran. Exception ini digunakan untuk menjaga invariant bahwa saldo uang pemain tidak boleh bernilai negatif. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- UangTidakCukupException() : NimonspoliException(1, "UANG_TIDAK_CUKUP"){};

2.22. Kelas SlotKartuPenuhException

Merupakan kelas turunan dari NimonspoliException untuk menangani kondisi ketika pemain mendapatkan kartu lagi saat slot kartu sudah penuh. Exception ini digunakan untuk menjaga invariant bahwa setiap pemain hanya boleh memiliki maksimal 3 kartu kemampuan spesial di tangannya. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- SlotKartuPenuhException() : NimonspoliException(2, "SLOT_KARTU_PENUH"){};

2.23. Kelas PerintahTidakDitemukanException

Merupakan kelas turunan dari NimonspoliException untuk menangani kondisi ketika user menjalankan perintah yang tidak dikenali sistem. Exception ini berguna untuk menjaga alur program berjalan semestinya. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- PerintahTidakDitemukanException() : NimonspoliException(3, "PERINTAH_TIDAK_DITEMUKAN"){};

2.24. Kelas FileTidakValidException

Merupakan kelas turunan dari NimonspoliException untuk menangani kondisi ketika file konfigurasi yang dibaca tidak valid, sehingga dapat mencegah kesalahan lebih lanjut dalam proses inisialisasi sistem. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- FileTidakValidException() : NimonspoliException(4, "FILE_TIDAK_VALID"){};

2.25. Kelas LogTransaksiEntry

Merupakan kelas yang merepresentasikan satu entri log dalam permainan. Setiap objek LogTransaksiEntry menyimpan informasi terkait suatu aksi pada permainan, seperti ronde, username pemain, jenis aksi yang dilakukan, serta detail dari aksi tersebut. Berikut daftar atribut dan metode.

- int ronde — nomor giliran/ronde
- string username — username pemain yang melakukan aksi
- string aksi — jenis aksi (misal: GERAK, PENJARA, KARTU, FESTIVAL, TELEPORT)
- string detail — detail lengkap aksi
- LogTransaksiEntry(int ronde, string username, string aksi, string detail) : konstruktor
- int getRonde() const

- const string& getUsername() const
- const string& getAksi() const
- const string& getDetail() const

2.26. Kelas LogTransaksiGame

Kelas ini menyimpan daftar objek LogTransaksiEntry sebagai riwayat aktivitas yang terjadi selama permainan berlangsung. Logger bekerja di latar belakang dan tidak memerlukan perintah khusus. Menyediakan mekanisme untuk menambahkan entri log dan mengaksesnya. Berikut daftar atribut dan metode.

- vector<LogTransaksiEntry> logs — daftar seluruh log
- void tambahLog(LogTransaksiEntry log) — menambahkan entri log
- const vector<LogTransaksiEntry>& getLogs() const — mengembalikan seluruh log
- vector<LogTransaksiEntry> getLogs(int top) const — mengembalikan sejumlah log terakhir
- LogTransaksiGame& operator<<(LogTransaksiEntry log) — operator overloading untuk menambahkan entri log secara sintaktis lebih ringkas

2.27. Kelas ConfigData

Kelas ini menyimpan informasi yang diperoleh dari berkas konfigurasi, di antaranya konfigurasi properti, konfigurasi fasilitas publik seperti railroad dan utility, konfigurasi pajak, konfigurasi petak spesial, serta konfigurasi lain-lain yang meliputi jumlah maksimal giliran bermain (maxTurn) dan saldo tiap pemain di awal permainan dimulai. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- map<int, PropertiConfig> propertiMap: key berupa id properti
- map<int, AksiConfig> aksiMap: key berupa id petak aksi
- map<int, int> hargaSewaRailroad: mapping jumlah railroad dengan biaya sewa
- map<int, int> pengaliUtility: mapping jumlah utility dengan faktor pengali
- int pphFlat
- int pphPersen
- int pbmFlat
- int gajiGo
- int dendaPenjara
- int maxTurn
- int uangAwalPemain
- Seluruh getter yang diperlukan untuk mengakses atribut konfigurasi

2.28. Kelas PropertiConfig

Kelas ini menyimpan informasi yang diperoleh dari seluruh berkas konfigurasi. Data yang disimpan meliputi: pemetaan properti (propertiMap), pemetaan petak aksi (aksiMap), harga sewa railroad, faktor pengali utility, parameter pajak, gaji petak GO, denda penjara, jumlah maksimal turn, dan uang awal pemain. Berikut daftar atribut dan metode.

- map<int, PropertiConfig> propertiMap — key: id properti
- map<int, AksiConfig> aksiMap — key: id petak aksi
- map<int, int> hargaSewaRailroad — mapping jumlah railroad -> biaya sewa
- map<int, int> pengaliUtility — mapping jumlah utility -> faktor pengali
- int pphFlat, pphPersen, pbmFlat — konfigurasi pajak
- int gajiGo — gaji saat melewati/mendarat di petak GO
- int dendaPenjara — denda untuk keluar penjara

- int maxTurn — maksimal jumlah giliran
- int uangAwalPemain — saldo awal setiap pemain
- Seluruh getter yang diperlukan untuk mengakses atribut konfigurasi

2.29. Kelas AksiConfig

Merupakan kelas yang merepresentasikan konfigurasi untuk satu properti. Data meliputi id, kode, nama, jenis (STREET/RAILROAD/UTILITY), warna, harga lahan, nilai gadai, harga bangun rumah, harga bangun hotel, dan vektor harga sewa untuk setiap level bangunan (L0 hingga L5). Berikut daftar atribut.

- int id, string kode, string nama, string jenis, string warna
- int hargaLahan, int nilaiGadai, int hargaBangunRumah, int hargaBangunHotel
- vector<int> hargaSewa — harga sewa level 0 s.d. 5

2.30. Kelas ConfigLoader

Merupakan kelas yang bertanggung jawab untuk membaca seluruh berkas konfigurasi dan mengonversinya menjadi objek ConfigData dan memastikan setiap nilai konfigurasi terisi dengan benar sesuai format yang ditentukan. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- ConfigData loadAll(string folderPath)
- map<int, PropertiConfig> loadProperty(vector<string> lines)
- map<int, AksiConfig> loadAksi(vector<string> lines)
- map<int, int> loadRailroad(vector<string> lines)
- map<int, int> loadUtility(vector<string> lines)
- TaxData loadTax(vector<string> lines): mengisi konfigurasi pajak (pphFlat, pphPersen, pbmFlat)
- SpecialData loadSpecial(vector<string> lines): mengisi konfigurasi petak spesial (gajiGo dan dendaPenjara)
- MiscData loadMisc(vector<string> lines): mengisi konfigurasi lain-lain (maxTurn dan uang awal pemain)

2.31. Kelas SaveNLoad

Merupakan kelas yang bertanggung jawab untuk menyimpan seluruh state permainan ke dalam file teks (.txt) serta memuat kembali state permainan dari file tersebut. Kelas ini menggunakan objek GameEngine sebagai sumber data saat proses penyimpanan, serta membangun kembali objek GameEngine saat proses pemuatan data.

Metode dari kelas ini terdiri dari :

- void saveGameState(GameEngine& game, string filename): menghasilkan file output teks yang menyimpan seluruh state permainan sesuai dengan format
- void loadGameState(GameEngine& game, string filename) : melakukan load state permainan dari file sehingga dapat melanjutkan permainan dari suatu state

2.32. Kelas FileHelper

Merupakan kelas helper yang menyediakan fungsi dasar untuk pembacaan dan penulisan file. Digunakan oleh kelas lain, seperti ConfigLoader dan SaveNLoad. Berikut adalah daftar lengkap terkait atribut dan metode yang digunakan.

- vector<string> readAllLines(string path): membaca setiap baris dari file kemudian menyimpannya dalam suatu list of string
- void writeAllLines(string path, vector<string> lines): menerima masukan list of string, kemudian menuliskannya ke file

2.33. Kelas GameUI

Merupakan kelas yang bertugas sebagai pengatur utama antarmuka pengguna (UI coordinator). Kelas ini tidak melakukan rendering secara langsung, tetapi mengoordinasikan berbagai komponen UI untuk menampilkan informasi ke layar.

Atribut dari kelas ini terdiri atas beberapa kelas komposisi:

- papanUI

- logUI
- pesanUI:
- pemainUI: PemainUI

Metode dari kelas ini terdiri dari :

- tampilkanPapan : menuliskan ke layar gambar dari papan
- tampilkanLog : menuliskan n teratas atau seluruh log permainan ke layar
- tampilkanPropertiPemain
- tampilkanAkta
- cetakPesan : untuk mencetak ke layar pesan kesalahan, atau informasi permainan

2.34. Kelas GameEngine

Merupakan kelas yang bertanggung jawab untuk menyimpan list dari semua player yang bermain, list dari semua petak dalam papan nimonspoli, dan urutan giliran player. Kelas ini juga bertanggung jawab untuk memulai permainan.

Atribut dari kelas ini terdiri atas :

- listPemain : Menyimpan list yang berisi semua pemain yang bermain pada suatu permainan nimonspoli
- papanPermainan : Menyimpan papan permainan yang berisi *state* masing-masing petak yang ada pada papan
- configData : Menyimpan konfigurasi data dari permainan
- dadu : Meyimpan dadu yang digunakan untuk permainan
- managerPenjara : Sebagai instans ManagerPenjara yang berguna untuk mengatur pemain-pemain yang berada di penjara
- managerProperti : Sebagai instans ManagerProperti yang mengatur properti-properti yang ada pada papanPermainan
- actionService : Sebagai instans dari PlayerActionService yang digunakan untuk mengatur aksi-aksi yang terjadi
- LogGame : Menyimpan log kejadian-kejadian yang terjadi dalam permainan nimonspoli

- Deck : Menyimpan tumpukan kartu spesial yang ada di dalam deck kartu
- urutanPemain : Menyimpan urutan pemain hasil pengacakan
- currentTurnIndex : Menyimpan
- maxTurn : Menyimpan jumlah turn maksimal pada suatu permainan
- arahNormalTurn : Menyimpan nilai boolean yang menunjukkan arah gerak pada permainan itu normal atau terbalik

Metode dari kelas ini terdiri dari :

- startGame : Metode untuk memulai suatu sesi permainan nimonspoli
- randomixeTurn : Metode untuk mengacak urutan bermain dari listPemain dan menyimpannya pada urutanPemain
- startTurn : Metode untuk menjalankan satu kali giliran pada seorang pemain

2.35. Kelas Dadu

Merupakan kelas yang bertanggung jawab untuk melakukan *roll* pengacakan pelemparan dadu untuk giliran jalan pemain.

Atribut dari kelas ini terdiri atas :

- d1 : Nilai dari dadu pertama yang dikocok
- d2 : Nilai dari dadu kedua yang dikocok

- mt19937 rng : generator bilangan acak yang digunakan untuk pengocokan dadu

Metode dari kelas ini terdiri dari :

- Dadu()
- void rollRandom(): metode untuk melakukan pengocokan dadu secara acak
- void rollManual(int v1, int v2): metode untuk mengatur nilai dadu secara manual
- int getNilaid1(): mendapatkan nilai dadu pertama
- int getNilaid2(): mendapatkan nilai dadu kedua
- int getTotalNilaiDadu(): mendapatkan total nilai dari kedua dadu

- bool isDouble(): mengecek apakah kedua dadu memiliki nilai yang sama

2.36. Kelas Kebangkrutan

Merupakan kelas yang bertanggung jawab untuk menangani kebangkrutan pemain. Kelas ini berguna untuk mendeklarasikan kebangkrutan seorang player dan menampilkan *interface* kebangkrutan untuk pemain tersebut.

Metode dari kelas ini terdiri dari :

- declareBangkrut : Metode untuk mendeklarasikan suatu player menjadi bangkrut dan mengeluarkannya dari permainan
- tampilkanBangkrut : Metode untuk menampilkan *message* kebangkrutan untuk pemain yang bangkrut

2.37. Kelas Likuidasi

Merupakan kelas yang bertanggung jawab untuk mengecek apakah pemain masih bisa bertahan dari kebangkrutan apabila masih memiliki aset yang dapat dilikuidasi. Apabila pemain masih mempunyai aset yang cukup, akan ditampilkan panel likuidasi untuk pemain tersebut hingga uangnya mencukupi untuk membayar yang harus dibayarnya agar tidak memasuki kebangkrutan.

Atribut dari kelas ini terdiri atas :

- config : Menyimpan informasi konfigurasi properti (PropertiConfig) yang diberikan kepada program untuk mendapatkan harga jual tanah dan bagunan yang dimiliki pemain

Metode dari kelas ini terdiri dari :

- bisaBayarDenganLikuidasi : menentukan apakah pemain yang berpotensi bangkrut masih bisa membayar biaya yang diperlukan dengan likuidasi aset-aset yang dimilikinya
- bisaBayarLangsung : menentukan apakah pemain sudah memiliki uang (cash) yang cukup untuk membayar biaya yang diperlukan
- totalLikuidasiSeluruhAsetGadai : menghitung total likuidasi seluruh aset pemain apabila seluruh asetnya digadaikan
- totalLikuidasiSeluruhAsetJual : menghitung total likuidasi seluruh aset pemain apabila seluruh asetnya dijual

- tampilkanPanelLikuidasi : menampilkan panel likuidasi kepada pemain yang berpotensi bangkrut (dalam panel ini,, pemain yang bersangkutan hanya bisa menggunakan *command* JUAL, GADAI, TEBUS, dan CETAK_PROPERTI)

2.38. Kelas SelesaiGame

Merupakan kelas yang bertanggung jawab untuk menangani selesainya suatu sesi permainan. Ada dua kemungkinan: MaxTurn (pemenang ditentukan dari total kekayaan, properti, lalu jumlah kartu) atau Bankruptcy (satu pemain tersisa). Berikut daftar metode.

- static Pemain* getPemenang(vector<Pemain*> listPemain) — mengembalikan pemenang sesuai ketentuan spek
- static bool sisaSatuOrang(vector<Pemain*> listPemain) — mengecek apakah hanya satu pemain aktif tersisa
- static void tampilPanelSelesaiKarenaMaxTurn(int maxTurn) — menampilkan pesan game selesai akibat batas turn
- static void tampilPanelSelesaiKarenaBangkrut() — menampilkan pesan game selesai akibat bankruptcy
- static void tampilkanPemenang(Pemain* p) — menampilkan antarmuka pemenang

2.39. Kelas Properti

Kelas Properti merupakan kelas yang bertanggung jawab untuk menangani berbagai **logika global terkait properti** yang melibatkan lebih dari satu objek properti. Kelas ini merupakan base class bagi seluruh jenis properti.

Atribut yang bersifat protected dari kelas ini terdiri atas:

- config : menyimpan informasi konfigurasi properti (PropertiConfig) yang diberikan kepada program yang terdiri atas id, kode, nama, jenis, warna, harga lahan, nilai gadai, harga bangun rumah, harga bangun hotel, dan tabel harga sewa.
- pemilik : menyimpan referensi (pointer) ke objek pemain yang memiliki properti tersebut. Jika properti belum dimiliki, maka bernilai nullptr.
- status : status kepemilikan properti yang terdiri atas BANK, OWNED, dan MORTGAGED, yang didefinisikan dalam enumerasi StatusProperti.

Adapun method-method yang bersifat publik dari kelas ini terdiri atas:

- Properti(const PropertiConfig& config) : konstruktor untuk menginisialisasi objek properti dengan parameter berupa konfigurasi dari properti. Secara default, properti belum dimiliki (pemilik = nullptr) dan memiliki status BANK.
- ~Properti() : destructor virtual yang digunakan untuk memastikan proses penghapusan objek turunan berjalan dengan benar dalam konteks polymorphism.
- hitungSewa(int dadu, ManagerProperti& manager) : method pure virtual yang harus diimplementasikan oleh setiap kelas turunan untuk menghitung nilai sewa berdasarkan jenis properti dengan parameter manager properti yang membantu dalam perhitungan dan nilai dadu yang digunakan khususnya pada properti tertentu seperti utility.
- gadai() : method untuk mengubah status properti menjadi MORTGAGED yang digunakan ketika pemain menggadaikan properti untuk mendapatkan sejumlah uang sesuai nilai gadai.
- batalGadai(): method untuk mengembalikan status properti dari MORTGAGED menjadi OWNED, biasanya setelah pemain membayar biaya penebusan.
- getKode() : mengembalikan informasi kode dari properti.
- getNamaProperti() : mengembalikan informasi nama properti.
- setPemilik(Pemain*) : mengatur pemilik dari properti.
- getPemilik() : prosedur yang dapat mengembalikan pemilik properti.
- getStatus() : mengembalikan status kepemilikan dari properti yang terdiri atas BANK, OWNED, atau MORTGAGED.

2.40. Kelas PropertiStreet

Kelas PropertiStreet merupakan turunan dari Properti yang merepresentasikan properti berupa lahan atau jalan yang dapat dibangun.

Kelas ini memiliki atribut tambahan yang bersifat private berupa:

- warna : kelompok warna dari properti yang digunakan untuk menentukan monopoli. Daftar kelompok warna disimpan di dalam enum kelas ColorGroup yang terdiri atas warna-warna petak yang terdefinisi sebagai kelompok-kelompok petak di papan permainan.
- level: menyatakan jumlah rumah yang telah dibangun pada properti untuk level 0-4 dan level 5 menyatakan bahwa properti sudah memiliki sebuah hotel dan merupakan level maksimal dari sebuah properti.
- tabelSewa : struktur data (vector) yang menyimpan nilai sewa berdasarkan tingkat pembangunan (tanpa bangunan, rumah 1–4, dan hotel).

Adapun method-method yang bersifat publik dari kelas ini terdiri atas:

- PropertiStreet(const PropertiConfig& config) : konstruktor yang menginisialisasi properti street dengan parameter configurasi properti.
- ~PropertiStreet() : destruktor yang memastikan pengancuran objek.
- hitungSewa(int dadu, ManagerProperti& manager) : method override yang menghitung nilai sewa berdasarkan kondisi properti, yaitu jumlah rumah atau keberadaan hotel. Nilai sewa diambil dari tabelSewa dan dapat dipengaruhi oleh kondisi lain seperti monopoli atau efek festival.
- bangun() : method untuk menambahkan satu unit rumah pada properti, dengan syarat validasi (seperti monopoli dan pemerataan pembangunan) telah dipenuhi oleh ManagerProperti.
- turunkanBangunan() : method yang dapat menurunkan level properti pada saat penjualan bangunan dilakukan.
- getWarna() : mengembalikan jenis warna dari properti street
- convertWarna(string warnaStr) : mengembalikan jenis warna yang diinputkan dalam tipe string menjadi bertipe ColorGroup
- getJumlahBangunan() : mengembalikan jumlah bangunan yang ada pada properti (0-5)
- getHargaBangun() : mengembalikan harga dari membangun properti

2.41. Kelas PropertiRailroad

Kelas PropertiRailroad merupakan turunan dari kelas Properti yang merepresentasikan properti berupa stasiun. Kelas ini tidak memiliki atribut tambahan, namun memiliki konstruktor yang digunakan untuk menginisialisasi objek properti dengan parameter berupa konfigurasi properti dan meneruskan inisialisasi tersebut ke konstruktor kelas induk.

Selain itu, kelas ini mengoverride method hitungSewa(int dadu, ManagerProperti& manager) dari kelas Properti. Method ini digunakan untuk menghitung nilai sewa berdasarkan jumlah properti railroad yang dimiliki oleh pemain. Perhitungan tersebut memanfaatkan informasi dari PropertyManager melalui method seperti hitungRailroad(). Dengan demikian, semakin banyak properti railroad yang dimiliki oleh pemain, maka semakin besar nilai sewa yang harus dibayarkan oleh pemain lain yang mendarat pada properti tersebut.

2.42. Kelas PropertiUtility

Kelas PropertiUtility merupakan turunan dari kelas Properti yang merepresentasikan properti utilitas seperti layanan publik (misalnya listrik atau air). Kelas ini tidak memiliki atribut tambahan, namun memiliki konstruktor yang digunakan untuk menginisialisasi objek properti dengan parameter berupa konfigurasi properti dan meneruskan inisialisasi tersebut ke konstruktor kelas induk.

Kelas ini juga mengoverride method hitungSewa(int dadu, ManagerProperti& manager) dari kelas Properti. Method ini menghitung nilai sewa berdasarkan hasil perkalian antara nilai mata dadu yang diperoleh pemain dengan faktor tertentu. Faktor pengali tersebut ditentukan berdasarkan jumlah properti utility yang dimiliki oleh pemain, yang diperoleh melalui PropertyManager menggunakan method seperti hitungUtility(). Dengan demikian, nilai sewa pada properti utility bersifat dinamis dan bergantung pada hasil dadu serta jumlah kepemilikan utilitas oleh pemain.

2.43. Kelas ManagerProperti

Kelas ManagerProperti bertanggung jawab untuk menangani berbagai logika global terkait properti yang melibatkan lebih dari satu objek properti dan juga membantu dan menyimpan nilai perhitungan pada properti.

Kelas ini memiliki atribut tambahan yang bersifat protected berupa:

- daftarProperti : menyimpan mapping id dengan pointer ke properti tertentu

- configData : konfigurasi data yang tersimpan

Beberapa method utama dari kelas ini yang bersifat publik meliputi:

- ManagerProperti(ConfigData configData) : konstruktor pembentuk objek manager properti berdasarkan konfigurasi data yang membentuk objek properti berdasarkan jenisnya, yakni street, railroad, dan utility
- getProperti(int id) : fungsi yang mengembalikan sebuah properti berdasarkan pengecekan idnya.
- getPropertiesByGroup(ColorGroup warna) : fungsi yang dapat mengembalikan daftar properti street dalam grup warna tertentu.
- isMonopoly(Pemain* pemain, ColorGroup group) : mengecek apakah seorang pemain memiliki seluruh properti dalam satu grup warna tertentu, dengan parameter berupa pointer ke pemain dan ColorGroup dengan return nilai boolean.
- isBisaBangun(StreetProperty* target) : memvalidasi apakah pemain dapat melakukan pembangunan pada properti, dengan mempertimbangkan kepemilikan penuh dan aturan pemerataan pembangunan dengan hasil bertipe boolean.
- isPunyaSemuaDalamGrup(Pemain* pemain, ColorGroup warna) : fungsi boolean yang mengembalikan true jika semua pemain di dalam kelompok warna tertentu sudah memiliki bangunan di petak warnanya.
- hitungRailroad(Pemain* pemain) : fungsi yang dapat menghitung jumlah properti railroad yang dimiliki oleh pemain.
- hitungUtility(Pemain* pemain) : fungsi yang dapat menghitung jumlah properti utility yang dimiliki oleh pemain.
- jualSemuaBangunan(PropertiStreet* properti) : method yang dapat menjual seluruh bangunan (rumah/hotel) pada properti tertentu, biasanya digunakan sebelum proses gadai.
- hitungLikuidasi(Pemain* pemain) : fungsi yang dapat menghitung total nilai aset pemain, termasuk properti dan bangunan, untuk keperluan evaluasi kebangkrutan.
- getHargaSewaRailroad(int jumlah) : fungsi yang mengembalikan harga sewa railroad dengan jumlah tertentu sesuai data pada configurasi.
- getPengaliUtility(int jumlah) : fungsi yang dapat mengembalikan nilai faktor pengali utility sesuai
- punyaSemuaDalamGrup(Pemain*, ColorGroup grup) : memeriksa apakah pemain memiliki semua properti dalam satu grup tertentu dengan return nilai boolean.

- getPropertiesByGroup(ColorGroup grup) : fungsi yang dapat mengembalikan daftar properti street dalam grup warna tertentu.
- ~ManagerProperti() : destruktor yang memastikan pemusnahan memori untuk objek tersebut terutama untuk array daftarProperti.

2.44. Kelas ManagerFestival

Kelas ManagerFestival bertanggung jawab untuk mengelola efek festival, yaitu peningkatan nilai sewa pada properti tertentu dalam jangka waktu terbatas. Efek festival disimpan dalam map<PetakProperti*, pair<int, int>> efekAktif yang memetakan setiap properti dengan nilai pengali sewa (1/2/4/8) dan sisa durasi efek (0-3). Pengali maksimum adalah 8 (3x penggandaan) dan durasi default adalah 3 giliran. Berikut daftar atribut dan metode.

- map<PetakProperti*, pair<int, int>> efekAktif — mapping properti ke {pengali, durasi}
- static constexpr int PENGALI_MAKS = 8
- static constexpr int DURASI_DEFAULT = 3
- void applyFestival(PetakProperti* properti) — menerapkan/memperkuat efek festival
- void updateDurasi(Pemain* pemilik) — mengurangi durasi efek setiap akhir giliran pemilik
- int getPengali(PetakProperti* properti) const — mengembalikan pengali sewa (1 jika tidak aktif)
- int getSisaDurasi(PetakProperti* properti) const
- bool isAktif(PetakProperti* properti) const
- string getFestivalString(PetakProperti* properti) const — representasi string untuk display
- void setEfekFestival(PetakProperti* properti, int pengali, int durasi) — untuk load game

2.45. Kelas Petak

Kelas Petak merupakan kelas abstrak yang menjadi basis bagi seluruh petak. Setiap subkelas dari kelas ini mendefinisikan perilakunya sendiri saat diinjak melalui diLahan(). Dikomposisi oleh kelas Papan dalam vector<Petak*>.

Atribut dari kelas ini terdiri atas:

- int indeks: posisi petak di papan (0–39)
- string kode: identifikasi singkat petak (e.g. "GO", "PEN")
- string nama: nama lengkap petak

Method dari kelas ini terdiri atas:

- Petak(int indeks, string kode, string nama) : konstruktor pembentuk objek petak berdasarkan indeks, kode, dan nama dari properti tersebut.
- getIndeks(): fungsi yang mengembalikan nilai indeks petak
- getKode(): fungsi yang mengembalikan kode petak
- getNama(): fungsi yang mengembalikan nama petak
- onLanded(Pemain* pemain, PlayerActionService* svc) (pure virtual): method pure virtual yang dipanggil secara polimorfik setiap kali pemain mendarat di petak ini yang akan diimplementasikan pada kelas-kelas turunannya.
- getDisplayInfo() : mengembalikan informasi ringkas petak untuk keperluan render
- ~Petak() : destruktor virtual yang memastikan objek terhapus dari memori dan di override oleh kelas-kelas turunannya.

2.46. Kelas PetakProperti

Kelas PetakProperti merupakan turunan dari Tile yang merepresentasikan petak papan yang berisi properti. Terdiri atas sebuah atribut protected Properti properti yang menyimpan properti.

Method dari kelas ini terdiri atas:

- PetakProperti(int indeks, string kode, string nama) : konstruktor pembentuk objek petak berdasarkan indeks, kode, dan nama dari properti tersebut yang juga memanggil kosntruktor kelas induknya.
- diLahan(Pemain* pemain, PlayerActionService* service) : method yang dipanggil ketika seorang pemain mendarat pada petak ini. Method ini tidak langsung melakukan aksi, tetapi akan mendelegasikan tindakan yang sesuai ke PlayerActionService, seperti membeli properti jika belum dimiliki atau membayar sewa jika dimiliki oleh pemain lain.
- ~PetakProperti() : destruktor yang memastikan objek terhapus dari memori yang merupakan override dari kelas parentnya

2.47. Kelas PetakAksi

Kelas abstrak turunan dari Petak yang menjadi basis bagi seluruh petak aksi non-properti. Berfungsi sebagai lapisan hierarki untuk mengelompokkan PetakSpesial, PetakKartu, PetakFestival, dan PetakPajak secara logis.

Method dari kelas ini terdiri atas:

- PetakAksi(int indeks, string kode, string nama) : konstruktor pembentuk objek petak berdasarkan indeks, kode, dan nama dari properti tersebut yang juga memanggil kosntruktor kelas induknya.
- onLanded(Pemain* pemain, PlayerActionService* svc) (pure virtual, override): method pure virtual yang implementasinya diteruskan ke kelas turunannya.
- ~PetakProperti() : destruktor yang memastikan objek terhapus dari memori yang merupakan override dari kelas parentnya

2.48. Kelas PetakSpesial

Kelas abstrak turunan dari PetakAksi yang menjadi basis bagi PetakMulai, PetakPenjara, PetakBebasParkir, dan PetakPergiKePenjara.

Method dari kelas ini terdiri atas:

- PetakSpesial(int indeks, string kode, string nama) : konstruktor pembentuk objek petak berdasarkan indeks, kode, dan nama dari properti tersebut yang juga memanggil kosntruktor kelas induknya.
- onLanded(Pemain* pemain, PlayerActionService* svc) (pure virtual, override): method pure virtual yang implementasinya diteruskan ke kelas turunannya.
- ~PetakSpesial() : destruktor yang memastikan objek terhapus dari memori yang merupakan override dari kelas parentnya

2.49. Kelas PetakMulai

Kelas turunan dari PetakSpesial yang merepresentasikan Petak Mulai Memberi gaji kepada pemain yang berhenti tepat di petak ini. Pemberian gaji saat pemain melewati petak ini ditangani oleh MovementController. Berinteraksi dengan Pemain (via +=) dan LogTransaksiGame.

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (pure virtual, override): menambahkan gaji ke saldo pemain melalui svc.transferMoney() dan mencatat ke LogTransaksiGame

2.50. Kelas PetakPenjara

Kelas turunan dari PetakSpesial yang merepresentasikan petak Penjara. Memiliki dua kondisi: pemain sebagai pengunjung atau tahanan. Seluruh logika detail penjara didelegasikan ke ManagerPenjara. Berinteraksi dengan ManagerPenjara dan LogTransaksiGame.

Atribut dari kelas ini terdiri atas:

- managerPenjara: referensi ke ManagerPenjara untuk mendelegasikan logika penjara

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override): Tidak melakukan apa-apa karena player yang lewat petak ini hanya akan lewat saja.

2.51. Kelas PetakBebasParkir

Kelas turunan dari PetakSpesial yang merepresentasikan petak Bebas Parkir. Petak ini berfungsi sebagai rest area tanpa aksi apa pun sehingga onLanded() bersifat no-op. Tetap diinstansiasi sebagai objek konkret agar polimorfisme pada Papan berjalan konsisten.

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override): tidak melakukan aksi apapun (no-op)

2.52. Kelas PetakPergiKePenjara

Kelas turunan dari PetakSpesial yang merepresentasikan petak Pergi ke Penjara. Saat diinjak, mendelegasikan pemindahan paksa pemain ke ManagerPenjara. Giliran pemain berakhir saat itu juga. Berkoordinasi dengan MovementController agar deteksi Mulai dinonaktifkan selama perpindahan paksa ini.

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override): memanggil masukkanKePenjara untuk memindahkan pemain ke penjara dan mengakhiri giliran.
- masukkanKePenjara(Pemain* pemain, PlayerActionService* svc): memasukan pemain ke penjara melalui *method* milik svc

2.53. Kelas PetakKartu

Kelas turunan dari PetakAksi yang merepresentasikan petak Kartu Kesempatan maupun Dana Umum. Saat diinjak, mengambil kartu teratas dari DeckKartu<KartuAksi*> dan mengeksekusinya secara polimorfik. Berinteraksi dengan DeckKartu<KartuAksi*> dan LogTransaksiGame.

Atribut dari kelas ini terdiri atas:

- jenis: membedakan antara petak Kesempatan atau Dana Umum

- deck: referensi ke DeckKartu<KartuAksi*> yang sesuai dengan jenis petak ini

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override) : memanggil ambilKartu() pada deck, memanggil jalankanEfek(pemain) secara polimorfik, lalu mengembalikan kartu via buangKartu()

2.54. Kelas PetakFestival

Kelas turunan dari PetakAksi yang merepresentasikan petak Festival. Saat diinjak, meminta pemain memilih properti miliknya lalu mendelegasikan penerapan efek ke ManagerFestival. Tidak mengelola logika festival secara langsung. Berinteraksi dengan ManagerFestival dan LogTransaksiGame.

Atribut dari kelas ini terdiri atas:

- managerFestival: referensi ke ManagerFestival sebagai dependensi eksternal

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override) : meminta pemain memilih properti, lalu memanggil applyFestival() pada ManagerFestival

2.55. Kelas PetakPajak

Kelas turunan dari PetakAksi yang merepresentasikan petak PPH dan PBM. Untuk PPH, pemain ditawarkan dua pilihan: nominal flat atau persentase dari total kekayaan (uang tunai + harga beli properti termasuk yang digadaikan + harga beli bangunan). Untuk PBM, pemain langsung dikenakan nominal flat. Besaran pajak diambil dari ConfigData

Atribut dari kelas ini terdiri atas:

- jenis: membedakan antara PPH atau PBM
- configData: referensi ke ConfigData untuk mengambil besaran pajak

Method dari kelas ini terdiri atas:

- onLanded(Pemain* pemain, PlayerActionService* svc) (override): menjalankan logika PPH atau PBM sesuai jenis, melempar UangTidakCukupException bila pemain tidak mampu membayar
- hitungTotalKekayaan(Pemain* pemain): menghitung total kekayaan pemain untuk opsi persentase PPH
- hitungPajak(Pemain* pemain) : menghitung nilai pajak yang harus dibayar. PPH mengembalikan nilai terkecil antara flat dan persentase sesuai pilihan pemain, PBM langsung mengembalikan nilai flat

2.56. Kelas Papan

Kelas yang mengkomposisi seluruh petak permainan dalam vector<Petak*>. Menyediakan akses ke petak berdasarkan indeks maupun kode. Dibangun saat inisialisasi permainan dengan menggunakan data ConfigReader, SaveNLoad, dan FileHelper. Berinteraksi dengan MovementController.

Atribut dari kelas ini terdiri atas:

- daftarPetak: vector<Petak*> berisi seluruh objek petak secara polimorfik

Method dari kelas ini terdiri atas:

- Papan(ManagerProperti manager, ConfigData configData) : konstruktor dari papan yang membuat objek berupa papan permainan berdasarkan configurasi petak dan informasi dari manager properti.
- getPetak(int indeks): mengakses petak berdasarkan posisi
- getPetak(string kode) (overload): mengakses petak berdasarkan kode
- findNearest(string kode, int indeks): mencari petak terdekat dari posisi indeks berdasarkan kode petaknya. Digunakan misalnya oleh efek kartu GoToNearestStationCard.
- getTotalPetak(): mengembalikan jumlah total petak pada papan.
- ~Papan() : destruktor yang memastikan pemusnahan memori untuk objek tersebut terutama untuk array daftarPetak.

2.57. Kelas MovementController

Kelas yang bertanggung jawab menghitung dan mengeksekusi perpindahan pemain. Mendeteksi lintasan PetakMulai untuk pemberian gaji. Memanggil onLanded() pada petak tujuan secara polimorfik.

Atribut dari kelas ini terdiri atas:

- papan: referensi ke Papan untuk menentukan petak tujuan
- configData: referensi ke ConfigData untuk besaran gaji Mulai

Method dari kelas ini terdiri atas:

- moveSteps(Pemain* pemain, int n): menghitung posisi baru secara modular melingkar 40 petak, mendeteksi petak Mulai untuk pemberian gaji via svc.transferMoney(), lalu memanggil onLanded(p, svc) pada petak tujuan
- teleport(Pemain* pemain, int idx): memindahkan pemain ke petak tertentu tanpa deteksi petak Mulai. Digunakan oleh TeleportCard dan LassoCard
- findNearestStation(Pemain* pemain): mencari petak stasiun terdekat dari posisi pemain saat ini, digunakan oleh efek GoToNearestStationCard

2.58. Kelas ManagerPenjara

Kelas yang mengelola seluruh logika kondisi penjara agar tidak tersebar di berbagai kelas. Dipanggil oleh PetakPenjara dan PetakPergiKePenjara. Berinteraksi dengan Pemain, Dadu, ConfigData, dan LogTransaksiGame.

Method dari kelas ini terdiri atas:

- tryEscapeByDouble(Pemain* pemain, Dadu* dadu, PlayerActionService* svc): memproses percobaan keluar penjara dengan melempar dadu double
- tryEscapeByFine(Pemain* pemain, PlayerActionService* svc): memproses keluar penjara dengan membayar denda via svc.transferMoney()
- paksaBayar(Pemain* pemain, PlayerActionService* svc): memaksa pembayaran denda pada percobaan ke-4

2.59. Kelas ManagerLelang

Merupakan kelas yang bertanggung jawab untuk mengelola proses lelang properti. Lelang dipicu saat pemain menolak/tidak mampu membeli properti street, atau saat pemain bangkrut ke Bank (semua propertinya dilelang). Lelang dimulai dari pemain setelah pemain pemicu, berlanjut sesuai urutan giliran, dan berakhir jika terdapat (jumlah_pemain - 1) pass berturut-turut. Berikut daftar atribut dan metode.

- ManagerTransaksi* managerTransaksi — referensi ke manajer transaksi
- explicit ManagerLelang(ManagerTransaksi* mt) : konstruktor
- void mulaiLelang(PetakProperti* properti, const vector<Pemain*>& daftarPemain, Pemain* pemainPemicu) — memulai proses lelang untuk properti tertentu

2.60. Kelas PlayerActionService

Kelas PlayerActionService merupakan kelas service yang mensentralisasi seluruh aksi atomik yang berulang dalam permainan. Kelas ini menjadi satu-satunya yang boleh memanggil MovementController dan ManagerPenjara secara langsung. Seluruh onLanded() di kelas Petak dan gunakan() di kelas kartu menerima referensi ke kelas ini sebagai parameter. Kelas ini juga menyimpan flag arahNormal dan currentTurnIdx untuk mendukung fitur ReverseCard. Berikut daftar atribut dan metode.

- MovementController* movementController
- Dadu* dadu — set dari luar via setDadu()
- ManagerPenjara* managerPenjara
- ManagerProperti* managerProperti
- ManagerFestival* managerFestival
- ManagerTransaksi* managerTransaksi

- ConfigData* config
- Papan* papan
- LogTransaksiGame* logger
- vector<Pemain*>* daftarPemain
- bool arahNormal — arah giliran bermain (true=normal, false=reverse)
- int* currentTurnIdx — indeks turn saat ini untuk mekanisme reverse
- void movePlayerRelative(Pemain& p, int n) — maju/mundur n petak (negatif = mundur)
- void teleportByInput(Pemain& p, const string& kodeOrIndeks) — teleport berdasarkan kode/indeks
- void moveToNearestStation(Pemain& p) — pindah ke stasiun terdekat
- void sendToJail(Pemain& p) — kirim pemain ke penjara
- void escapeFromJailByCard(Pemain& p) — bebaskan pemain dari penjara via kartu
- void jailOpponent(Pemain& p) — memilih dan memenjarakan pemain lawan (PenjaraKanCard)
- void transferMoney(Pemain* asal, Pemain* tujuan, int jumlah) — nullptr = ke/dari Bank
- void collectFromAllPlayers(Pemain& penerima, int jumlahPerOrang) — KartuHariUlangTahun
- void payToAllPlayers(Pemain& pembayar, int jumlahPerOrang) — KartuMauNyaleg
- void bayarSewa(Pemain& penyewa, PetakProperti& properti, int nilaiDadu)
- void beliProperti(Pemain& pemain, PetakProperti& properti)
- void gadaiProperti(Pemain& pemain)
- void tebusProperti(Pemain& pemain)
- void bangunProperti(Pemain& pemain)

- void demolishOpponentProperty(Pemain& pemain) — DemolitionCard
- void pullPlayerAhead(Pemain& pemain) — LassoCard
- void rotateAllHandCards(Pemain& pemain) — RotasiKartuCard
- void reverseTurnOrder(Pemain& pemain) — ReverseCard
- void beriSemuaAset(Pemain* asal, Pemain* tujuan) — transfer aset saat kebangkrutan
- void logFestivalActivation(Pemain& pemain, PetakProperti& prop, int pengali, int durasi)
- void logAksi(Pemain& pemain, const string& aksi, const string& detail)
- bool isArahNormal() const : getter arahNormal
- const vector<Pemain*>& getDaftarPemain() const
- ConfigData* getConfig() : getter config
- void setDadu(Dadu* d) : setter dadu
- Dadu* getDadu() const : getter dadu

2.61. Kelas OutputHandler

Merupakan kelas yang bertanggung jawab menangani seluruh output ke layar (UI Layer). Kelas ini menyediakan berbagai metode statis untuk menampilkan papan permainan, akta properti, properti pemain, panel gadai/tebus/bangun, serta pesan umum dan error. Berikut daftar metode utama.

- static void cetakPesan(const string& pesan)
- static void cetakError(const string& errorMsg)
- static void cetakMendaratDiPetak(const string& namaPemain, const string& namaPetak)
- static void cetakEfekKartu(const string& namaKartu, const string& pesan)
- static void cetakTransaksi(const string& pembayar, int jumlah, const string& penerima)
- static void cetakAksi(const string& namaPemain, const string& aksi)

- static void cetakAktaProperti(PetakProperti* properti, const ConfigData& config)
- static void cetakPapan(const Papan& papan, const vector<Pemain*>& daftarPemain, int currentTurn, int maxTurn)
- static void cetakPropertiPemain(const Pemain* pemain, const ConfigData& config)
- static void tampilkanPropertiYangBisaDigadai(Pemain* pemain, const ConfigData& config)
- static void tampilkanPropertiYangBisaDitebus(Pemain* pemain, const ConfigData& config)
- static void tampilkanPropertiYangBisaDibangun(Pemain* pemain, ManagerProperti& manager, const ConfigData& config)
- Private helpers: getColorCode(), resetColor(), getBangunanSimbol(), findPropertiConfig()

2.62. Kelas InputHandler

Merupakan kelas yang bertanggung jawab menangani seluruh input dari pengguna dengan validasi. Menyediakan metode statis untuk meminta input berulang hingga valid. Berikut daftar metode.

- static int promptAngka(const string& pesan, int min, int max) — meminta input angka dalam rentang [min, max]
- static string promptString(const string& pesan) — meminta input satu kata/token
- static string promptLine(const string& pesan) — meminta input satu baris penuh
- static int mintaPilihanPajak(int pphFlat, int pphPersen) — prompt khusus pilihan pajak PPH

2.63. Kelas GameConstants

Merupakan kelas yang menyimpan konstanta-konstanta global permainan yang tidak bergantung pada konfigurasi file. Digunakan sebagai sumber tunggal kebenaran untuk nilai-nilai yang sering digunakan lintas kelas. Berikut daftar konstanta.

- static const int DENDA_PENJARA — denda keluar penjara (default, dapat di-override config)
- static const int MAX_PERCOBAAN_PENJARA — maksimal percobaan keluar penjara dengan dadu
- static const int SALDO_AWAL_DEFAULT — saldo awal pemain (default)
- static constexpr const char* LOG_AKSI_GERAK = "GERAK"
- static constexpr const char* LOG_AKSI_TELEPORT = "TELEPORT"

- static constexpr const char* LOG_AKSI_PENJARA = "PENJARA"
- static constexpr const char* LOG_AKSI_KARTU = "KARTU"
- static constexpr const char* LOG_AKSI_FESTIVAL = "FESTIVAL"

3. Justifikasi

Desain kelas pada permainan Nimonspoli ini disusun dengan menerapkan konsep *Object Oriented Programming*, yang meliputi enkapsulasi, inheritance, dan polymorphism. Perancangan ini dilakukan untuk membagi tanggung jawab sistem ke dalam kelas-kelas yang spesifik berdasarkan karakteristik dan fungsinya serta berbentuk modular. Rancangan kelas ini sudah menangani keseluruhan kebutuhan pengembangan program sesuai spesifikasi tugas, mulai dari kelas-kelas yang berhubungan dengan inisialisasi permainan, tampilan permainan, penanganan file config, save & load, hingga logika permainan yang dirancang secara detail. Desain kelas ini bertujuan untuk memisahkan setiap tanggung jawab kelas sehingga meminimalkan ketergantungan antarkomponen.

Inheritance dan polymorphism dilakukan pada kelas-kelas yang memiliki tanggung jawab besar seperti Petak, Properti, dan Kartu, yang menjadi base class sehingga berbagai jenis metode dapat diimplementasikan dengan penyesuaian yang dibutuhkan pada kelas turunannya. Selain itu, untuk tugas-tugas yang melibatkan perhitungan juga dipisahkan menjadi kelas-kelas manager seperti ManagerGiliran, ManagerPenjaga, ManagerProperti, dan ManagerFestival. Hal ini dilakukan guna menghindari penumpukkan tanggung jawab pada satu kelas tertentu. Perancangan kelas ini juga menggunakan generic class pada deckKartu, sehingga dapat menangani berbagai jenis kartu tanpa perlu melakukan pengulangan pendefinisian pada berbagai jenis kartu

Pada perancangan kelas ini, terdapat beberapa kelebihan dan kekurangan yang kami evaluasi. Adapun kelebihan dari desain kelas yang sudah dirancang yakni, pemodelan kelas yang sudah cukup spesifik dan modular, hubungan antarkelas yang terstruktur dengan menyesuaikan konsep polymorphism, serta sudah tidak terdapat redundansi tanggung jawab antarkelas. Adapun kekurangan yang masih mungkin ada pada perancangan kelas ini yakni cukup banyaknya kelas-kelas yang dihasilkan dari analisis perancangan ini sehingga memungkinkan peningkatan kompleksitas pengerjaan implementasi program pada milestone berikutnya. Selain itu, masih

dimungkinkan terjadinya kekeliruan dari pemisahan tanggung jawab ataupun ketidaklengkapan atribut maupun metode yang dirancang yang mungkin akan lebih terlihat ketika pengerjaan implementasi kode nantinya.

Selama pengerjaan perancangan desain kelas ini, kami mengalami kendala dalam menentukan desain yang tepat sesuai penjelasan deskripsi spesifikasi tugas yang diberikan agar dapat menyesuaikan konsep OOP. Selain itu, pembagian tanggung jawab setiap kelas yang harus ditentukan dengan baik agar tidak terjadi overlap ataupun ketidaklengkapan pada masing-masing kelas. Komunikasi pada pembagian kelas juga cukup membingungkan untuk dilakukan karena masih terdapat perbedaan pemahaman antar anggota di awal pengerjaan. Akan tetapi semua kesulitan ini sudah kami tangani dengan baik selama pengerjaan sehingga tugas perancangan kelas dapat selesai tepat waktu dan sesuai dengan instruksi pengerjaan.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Nama File: include/models/Kartu

```
class Kartu {
public:
    virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0;
};

class KartuKesempatan : public Kartu {
public:
    virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0;
};

class KartuDanaUmum : public Kartu {
```

```
public:
    virtual void jalankanEfek(Pemain& pemain, PlayerActionService& svc) = 0;
};

class KartuPergiKeStasiunTerdekat : public KartuKesempatan {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuMundurTigaPetak : public KartuKesempatan {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuMasukPenjara : public KartuKesempatan {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuBebasPenjara : public KartuKesempatan {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuHariUlangTahun : public KartuDanaUmum {
public:
```

```
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuBiayaDokter : public KartuDanaUmum {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuMauNyaleg : public KartuDanaUmum {
public:
    void jalankanEfek(Pemain& pemain, PlayerActionService& svc) override;
};

class KartuKemampuanSpesial {
public:
    virtual void gunakan(Pemain& pemain, PlayerActionService& svc) = 0;
    virtual std::string getDeskripsi() const = 0;
};

class MoveCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class DiscountCard : public KartuKemampuanSpesial {
```

```
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class ShieldCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class TeleportCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class LassoCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class DemolitionCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
```

```
    std::string getDeskripsi() const override;
};


// kartu bonus

class RotasiKartuCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class ReverseCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};

class PenjaraKanCard : public KartuKemampuanSpesial {
public:
    void gunakan(Pemain& pemain, PlayerActionService& svc) override;
    std::string getDeskripsi() const override;
};
```

Bagian kartu menerapkan dua hierarki inheritance yang terpisah sesuai peran masing-masing jenis kartu dalam permainan. Hierarki pertama adalah hierarki Kartu, dengan kelas abstrak Kartu yang mendefinisikan pure virtual function jalankanEfek(Pemain&, PlayerActionService&). Kartu kemudian memiliki dua kelas intermediate abstract yakni KartuKesempatan dan KartuDanaUmum yang masing-masingnya diturunkan kembali menjadi kelas konkret yakni KartuPergiKeStasiunTerdekat, KartuMundurTigaPetak, KartuMasukPenjara, dan KartuBebasPenjara dari KartuKesempatan serta KartuHariUlangTahun, KartuBiayaDokter, KartuMauNyaleg dari KartuDanaUmum. Hierarki kedua seputar kartu yakni KartuKemampuanSpesial dengan kelas abstrak KartuKemampuanSpesial yang mendefinisikan dua pure virtual function yakni gunakan() dan getDeskripsi(). Dari KartuKemampuanSpesial, terdapat enam kelas turunan konkret yakni MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard. Pemisahan Kartu dan KartuKemampuanSpesial dilakukan karena KartuKemampuanSpesial memiliki mekanisme yang berbeda dengan Kartu, yakni dapat disimpan di tangan pemain dan mampu digunakan kapan saja sebelum dadu. Dengan pemisahan ini, desain menjadi lebih modular dan mudah dipahami karena setiap hierarki menangani satu jenis perilaku (*single responsibility*). Penggunaan kelas abstrak dan fungsi virtual pun memungkinkan penerapan polymorphism, sehingga method onLanded() milik PetakKartu cukup memanggil jalankanEfek(pemain, svc) tanpa perlu mengetahui jenis kartu sebenarnya.

Nama File: include/models/Petak

```
class Petak{
protected:
    int indeks;
    std::string kode;
    std::string nama;
public:
    virtual void onLanded(Pemain& pemain, PlayerActionService& actionService) = 0;
};
```

```
class PetakAksi : public Petak{
public:
    virtual void onLanded(Pemain& pemain, PlayerActionService& actionService) = 0;
};

class PetakProperti : public Petak {
public:
    virtual void onLanded(Pemain& pemain, PlayerActionService& actionService);
};

class PetakFestival : public PetakAksi {
public:
    void onLanded(Pemain& pemain, PlayerActionService& svc) override;
};

class PetakKartu : public PetakAksi {
public:
    void onLanded(Pemain& pemain, PlayerActionService& svc) override;
};
class PetakSpesial : public PetakAksi {
public :
    virtual void onLanded (Pemain &p, PlayerActionService actionService) = 0;
};

class PetakMulai : public PetakSpesial {
```

```
public :
    void onLanded (Pemain &p, PlayerActionService actionService) override;
};

class PetakPenjara : public PetakSpesial {
public :
    void onLanded (Pemain &p, PlayerActionService actionService) override;
};

class PetakPergiPenjara : public PetakSpesial {
public :
    void onLanded (Pemain &p, PlayerActionService actionService) override;
};
class PetakBebasParkir : public PetakSpesial {
public :
    void onLanded (Pemain &p, PlayerActionService actionService) override;
};
```

Pada sistem petak permainan, konsep inheritance (pewarisan) digunakan dengan menjadikan kelas Petak sebagai base class yang merepresentasikan satu petak pada papan permainan. Kelas ini kemudian diturunkan menjadi beberapa kelas turunan seperti PetakAksi dan PetakProperti, serta diturunkan lebih lanjut menjadi kelas-kelas spesifik seperti PetakKartu, PetakFestival, dan berbagai jenis PetakSpesial (misalnya PetakMulai, PetakPenjara, dan PetakBebasParkir). Dengan struktur ini, setiap jenis petak dapat memiliki atribut dan perilaku khusus tanpa mengubah struktur dasar dari kelas Petak. Selain itu, konsep polymorphism juga diterapkan melalui method onLanded yang dioverride oleh setiap subclass, sehingga setiap jenis petak dapat menentukan perilakunya masing-masing tanpa perlu melakukan pengecekan tipe secara manual seperti if-else atau switch. Hal ini membuat kode menjadi lebih modular, mudah

dikembangkan (misalnya dalam penambahan jenis petak baru), serta lebih mudah dipelihara karena perubahan pada satu jenis petak tidak memengaruhi yang lain.

Nama File: include/models/Properti

```
class Properti{
protected:
    const PropertiConfig* config;
    Pemain* pemilik;
    StatusProperti status;

public:
    virtual int hitungSewa(int dadu, ManagerProperti& manager) = 0;
};

class PropertiRailroad : public Properti{
public:
    int hitungSewa(int dadu, ManagerProperti& manager) override;
};

class PropertiStreet : public Properti{
public:
    int hitungSewa(int dadu, ManagerProperti& manager) override;
};
```

```
class PropertiUtility : public Properti{
public:
    int hitungSewa(int dadu, ManagerProperti& manager) override;
};
```

Pada sistem properti dalam permainan, konsep inheritance (pewarisan) digunakan dengan menjadikan kelas Properti sebagai base class yang merepresentasikan entitas umum dari semua jenis properti. Kelas ini kemudian diturunkan menjadi beberapa kelas turunan seperti PropertiStreet, PropertiRailroad, dan PropertiUtility, yang masing-masing merepresentasikan jenis properti dengan aturan berbeda. Setiap kelas turunan mengimplementasikan perilaku spesifiknya sendiri, terutama melalui method hitungSewa yang dideklarasikan sebagai pure virtual function pada kelas Properti. Hal ini menunjukkan penerapan polymorphism, di mana pemanggilan method hitungSewa pada objek bertipe Properti akan menghasilkan perilaku yang berbeda tergantung jenis properti sebenarnya, tanpa perlu melakukan pengecekan tipe secara manual. Dengan pendekatan ini, perhitungan sewa untuk setiap jenis properti dapat disesuaikan dengan aturan masing-masing, sehingga kode menjadi lebih modular, mudah dikembangkan, dan menghindari duplikasi logika perhitungan.

Nama File: include/utils/NimonspoliException.hpp

```
class NimonspoliException : public std::exception {
public:
    NimonspoliException(int kode, std::string tipe) : kodeError(kode), tipeError(tipe) {}
    virtual const char* what() const noexcept override { return tipeError.c_str(); }
    . . .
    . . .
```

```
}

class UangTidakCukupException : public NimonspoliException{
public:
    UangTidakCukupException() : NimonspoliException(1, "UANG_TIDAK_CUKUP"){};
};

class SlotKartuPenuhException : public NimonspoliException{
public:
    SlotKartuPenuhException() : NimonspoliException(2, "SLOT_KARTU_PENUH"){};
};

class PerintahTidakDitemukanException : public NimonspoliException{
public:
    PerintahTidakDitemukanException() : NimonspoliException(3, "PERINTAH_TIDAK_DITEMUKAN"){};
};

class FileTidakValidException : public NimonspoliException{
public:
    FileTidakValidException() : NimonspoliException(4, "FILE_TIDAK_VALID"){};
};
```

Pada sistem penanganan error, konsep inheritance digunakan dengan menjadikan kelas NimonspoliException sebagai base class yang merupakan turunan dari std::exception. Kelas ini kemudian diturunkan menjadi beberapa kelas spesifik seperti

UangTidakCukupException, SlotKartuPenuhException, PerintahTidakDitemukanException, dan FileTidakValidException, yang masing-masing merepresentasikan jenis kesalahan tertentu dalam permainan. Setiap kelas turunan menginisialisasi nilai kodeError dan tipeError melalui constructor parent, sehingga memungkinkan representasi error yang konsisten. Selain itu, konsep polymorphism diterapkan melalui method what() yang di-*override* pada kelas NimonspoliException dari std::exception. Method ini kemudian digunakan oleh seluruh kelas turunannya untuk mengembalikan pesan error yang sesuai berdasarkan nilai tipeError, tanpa perlu melakukan override ulang pada setiap subclass. Pendekatan ini membuat sistem penanganan error menjadi lebih terstruktur, mudah diperluas dengan menambahkan jenis exception baru, serta meningkatkan keterbacaan dan konsistensi dalam penanganan kesalahan.

4.2. Method/Operator Overloading

Nama File: src/models/Pemain.cpp

```
// operator overload
Pemain& Pemain::operator+=(int amount){
    saldo += amount;
    return *this;
}
Pemain& Pemain::operator-=(int amount){
    if (saldo < amount) throw UangTidakCukupException();

    saldo -= amount;
    return *this;
}
bool Pemain::operator>(const Pemain& other) const{
    //  Pemenang adalah pemain dengan uang terbanyak.
    //  Jika seri, pemain yang memiliki jumlah petak properti terbanyak yang menjadi pemenang.
```

```
    //  Jika seri, pemain yang memiliki jumlah kartu terbanyak yang menjadi pemenang.
    //  Jika seri, semua pemain yang masih seri menjadi pemenang.
    if (saldo != other.saldo) return saldo > other.saldo;
    if (asetPemain.size() != other.asetPemain.size()) return asetPemain.size() > other.asetPemain.size();
    if (kartuDiTangan.size() != other.kartuDiTangan.size()) return kartuDiTangan.size() > other.kartuDiTangan.size();
    return false;
}
bool Pemain::operator<(const Pemain& other) const{
    if (saldo != other.saldo) return saldo < other.saldo;
    if (asetPemain.size() != other.asetPemain.size()) return asetPemain.size() < other.asetPemain.size();
    if (kartuDiTangan.size() != other.kartuDiTangan.size()) return kartuDiTangan.size() < other.kartuDiTangan.size();
    return false;
```

Pada kelas Pemain, digunakan konsep operator overloading untuk mempermudah manipulasi objek pemain, khususnya dalam operasi yang sering dilakukan seperti perubahan saldo dan perbandingan antar pemain. Operator yang dioverload meliputi: (1) operator+= untuk menambahkan saldo pemain; (2) operator-= untuk mengurangi saldo pemain; (3) operator> untuk membandingkan dua pemain pada akhir permainan untuk menentukan pemenang; (4) operator< untuk membandingkan dua pemain pada akhir permainan untuk menentukan pemenang. Alasan digunakannya operator overloading adalah membuat kode lebih natural dan intuitif sehingga meningkatkan keterbacaan kode (*readability*).

Nama File: src/utils/LogTransaksiGame.cpp

```
LogTransaksiGame& LogTransaksiGame::operator<<(LogTransaksiEntry log) {
    tambahLog(move(log));
    return *this;
```

```
}
```

Pada kelas LogTransaksiGame, digunakan konsep operator overloading melalui operator<< untuk menyederhanakan proses penambahan entri log ke dalam daftar. Operator ini memungkinkan objek log ditambahkan dengan sintaks yang lebih natural seperti logGame << entry, yang secara internal memanggil metode tambahLog. Pendekatan ini meningkatkan keterbacaan kode dan membuat penggunaan kelas lebih intuitif dibandingkan pemanggilan fungsi biasa.

Nama File: include/models/ConfigData.hpp

```
const std::map<int, PropertiConfig> &getPropertiMap() const;
const PropertiConfig &getPropertiMap(const int &idProperti) const;

const std::map<int, AksiConfig> &getAksiMap() const;
const AksiConfig &getAksiMap(const int &idAksi) const;

const std::map<int, int> &getHargaSewaRailroad() const;
int getHargaSewaRailroad(int jumlahRailroad) const;

const std::map<int, int> &getPengaliUtility() const;
int getPengaliUtility(int jumlahUtility) const;
```

Pada kelas ConfigData, digunakan konsep method overloading pada beberapa fungsi getter seperti getPropertiMap, getAksiMap, getHargaSewaRailroad, dan getPengaliUtility. Setiap method memiliki nama yang sama namun parameter yang berbeda,

sehingga dapat digunakan untuk dua kebutuhan sekaligus, yaitu mengambil seluruh data dalam bentuk map atau mengambil nilai spesifik berdasarkan key tertentu. Contohnya, getPropertiMap() mengembalikan seluruh map properti, sedangkan getPropertiMap(int idProperti) mengembalikan satu objek PropertiConfig berdasarkan id. Penggunaan method overloading ini membuat antarmuka kelas lebih ringkas dan intuitif, karena pengguna cukup mengingat satu nama fungsi untuk dua operasi yang berkaitan, sekaligus meningkatkan fleksibilitas dalam mengakses data konfigurasi.

Nama File: include/models/ConfigData.hpp

```
const std::vector<LogTransaksiEntry>& getLogs() const { return logs; }
std::vector<LogTransaksiEntry> getLogs(int top) const;
```

Pada kelas LogTransaksiGame, digunakan konsep method overloading pada fungsi getLogs, yaitu getLogs() dan getLogs(int top). Kedua method memiliki nama yang sama tetapi parameter berbeda, sehingga memberikan fleksibilitas dalam pengambilan data log. Method getLogs() digunakan untuk mengambil seluruh riwayat log yang tersimpan, sedangkan getLogs(int top) digunakan untuk mengambil sejumlah entri log terakhir sesuai dengan parameter yang diberikan. Implementasi getLogs(int top) memanfaatkan slicing pada vector untuk mengambil bagian akhir dari log secara efisien. Penggunaan method overloading ini membuat antarmuka kelas lebih intuitif dan memudahkan pengguna dalam memilih kebutuhan akses data tanpa harus menggunakan nama fungsi yang berbeda.

Nama File: include/models/Papan.hpp

```
Petak* getPetak(int indeks);
Petak* getPetak(const std::string& kode);
```

Pada kelas Papan, digunakan konsep method overloading pada fungsi getPetak, yaitu getPetak(int indeks) dan getPetak(string kode). Kedua method memiliki nama yang sama namun parameter berbeda, sehingga memungkinkan pengambilan objek petak dengan dua cara, yaitu berdasarkan posisi indeks pada papan atau berdasarkan kode unik petak. Method getPetak(int indeks) digunakan untuk mengakses petak berdasarkan urutan pada papan permainan, sedangkan getPetak(string kode) melakukan pencarian petak secara iteratif berdasarkan kode. Penggunaan method overloading ini meningkatkan fleksibilitas dan kemudahan penggunaan kelas, karena pengguna dapat memilih cara akses yang paling sesuai tanpa perlu menggunakan nama fungsi yang berbeda.

4.3. Template & Generic Classes

Nama File: include/models/Kartu/DeckKartu.hpp

```
template <typename T>
class DeckKartu {
private:
    std::vector<T*> tumpukanKartu; // sumber kartu yang dapat diambil
    std::vector<T*> pembuangan; // kartu yang telah digunakan
    . . .
    . . .
```

```
}
```

DeckKartu<T> adalah generic class template yang berperan untuk mengelola tumpukan dan pembuangan kartu untuk sembarang tipe kartu T. Dalam permainan ini, class ini diinstansiasi dua kali yakni pada DeckKartu<Kartu> untuk deck Kesempatan dan deck Dana Umum, serta DeckKartu<KartuKemampuanSpesial> untuk deck Skill. Keuntungan dari penggunaan generic class DeckKartu<T> pada permainan ini adalah dari segi keamanan, DeckKartu<Kartu> tidak bisa tidak sengaja menerima KartuKemampuanSpesial begitu pula sebaliknya. Tanpa template, masing-masing deck dari berbagai jenis kartu harus didefinisikan secara langsung, yang melanggar prinsip DRY.

4.4. Exception

Nama File: src/models/Pemain.cpp & src/utils/ConfigLoader.cpp

```
Pemain& Pemain::operator-=(int amount){
    if (saldo < amount) throw UangTidakCukupException();

    saldo -= amount;
    return *this;
}

std::vector<std::string> FileHelper::readAllLines(const std::string& path){
    std::ifstream file(path);

    if (!file.is_open()) throw FileTidakValidException();
    . . .
```

```
  . . .
}
```

Pada sistem Nimonspoli, digunakan konsep exception handling untuk menangani kondisi kesalahan yang dapat melanggar aturan permainan (invariant), seperti pemain tidak memiliki uang yang cukup atau jumlah kartu melebihi batas maksimum. Exception direpresentasikan melalui kelas NimonspoliException sebagai base class yang merupakan turunan dari std::exception, serta beberapa kelas turunan seperti UangTidakCukupException, SlotKartuPenuhException, PerintahTidakDitemukanException, dan FileTidakValidException yang merepresentasikan jenis kesalahan spesifik. Pendekatan ini membuat sistem penanganan error menjadi lebih terstruktur, mudah diperluas dengan menambahkan jenis exception baru, serta meningkatkan keterbacaan dan konsistensi dalam penanganan kesalahan.

4.5. C++ Standard Template Library

Nama File: include/models/Kartu/DeckKartu.hpp & include/models/Papan.hpp & include/models/Pemain.hpp & include/models/Petak/PetakFestival.hpp & include/models/ConfigData.hpp

```
class DeckKartu {
private:
   std::vector<T*> tumpukanKartu;
   std::vector<T*> pembuangan;
}
class Papan{
private:
```

```
    std::vector<Petak*> daftarPetak;
}
class Pemain {
private:
    std::vector<KartuKemampuanSpesial*> kartuDiTangan;
    std::vector<Properti*> asetPemain;
}
class ManagerFestival {
private:
    std::map<Properti*, std::pair<int, int>> efekAktif;
}
class ConfigData{
private:
    std::map<int, PropertiConfig> propertiMap; //untuk setiap prop id <-> its detail
    std::map<int, AksiConfig> aksiMap; //untuk setiap petak aksi id <-> its detail
    std::map<int, int> hargaSewaRailroad; //untuk setiap jumlahRailroad
    std::map<int, int> pengaliUtility; //untuk setiap jumlahUtility
}
```

Pada permainan ini STL vector digunakan pada beberapa bagian implementasi kode, diantaranya yakni std::vector<T*> yang digunakan di DeckKartu<T> untuk tumpukanKartu dan pembuangan, std::vector<Petak*> yang digunakan pada daftarPetak untuk menyimpan petak pada papan permainan. Penggunaan STL vector ini memungkinkan dipanggilnya push_back dan pop_back untuk mekanisme penambahan dan pengurangan kartu di dalam deck dan pembentukkan susunan papan secara efisien. Pemanggilan std::shuffle pun dilakukan untuk mekanisme pengacakan kartu. Pada mekanisme Pemain, std::vector<KartuKemampuanSpesial*> digunakan untuk kartuDiTangan dengan akses *index-based*. STL ini digunakan oleh pemain untuk menampung

KartuKemampuanSpesial di tangan pemain dan memilih kartu yang hendak dibuang. Selain vector, map pun digunakan di ManagerFestival untuk menyimpan pasangan antara pengali dan durasi dari setiap properti, juga pada beberapa atribut pada ConfigData. Map dipilih karena *lookup* berdasarkan suatu keyword unik haruslah efisien.

Nama File: src/models/ConfigData.cpp

```
ConfigData::ConfigData(int pphFlat, int pphPersen, int pbmFlat, int gajiGo, int dendaPenjara,
                       int maxTurn, int uangAwal, map<int, PropertiConfig> propertiMap, map<int, AksiConfig> aksiMap,
                       map<int, int> sewaRailroad, map<int, int> pengaliUtility)
    : propertiMap(move(propertiMap)), aksiMap(move(aksiMap)), hargaSewaRailroad(move(sewaRailroad)),
      pengaliUtility(move(pengaliUtility)), pphFlat(pphFlat), pphPersen(pphPersen), pbmFlat(pbmFlat),
      gajiGo(gajiGo), dendaPenjara(dendaPenjara), maxTurn(maxTurn), uangAwalPemain(uangAwal) {}

PropertiConfig::PropertiConfig(int id, string kode, string nama, string jenis, string warna, int hargaLahan,
                               int nilaiGadai, int hargaBangunRumah, int hargaBangunHotel, vector<int> hargaSewa)
    : id(id), kode(move(kode)), nama(move(nama)), jenis(move(jenis)), warna(move(warna)),
      hargaLahan(hargaLahan), nilaiGadai(nilaiGadai), hargaBangunRumah(hargaBangunRumah),
      hargaBangunHotel(hargaBangunHotel), hargaSewa(move(hargaSewa)) {}
```

Digunakan juga fungsi std::move pada beberapa bagian. Fungsi ini digunakan untuk mengimplementasikan konsep move semantics, yaitu memindahkan sumber daya dari suatu objek ke objek lain tanpa melakukan penyalinan (copy). Contohnya pada constructor PropertiConfig dan ConfigData, parameter seperti string, vector, dan map dipindahkan menggunakan move, sehingga lebih efisien dalam penggunaan memori dan waktu eksekusi, terutama ketika menangani struktur data berukuran besar. Penggunaan std::move ini menghindari overhead yang terjadi pada proses copy, sehingga performa program menjadi lebih optimal.

Nama File: src/utils/LogTransaksiGame.cpp

```
vector<LogTransaksiEntry> LogTransaksiGame::getLogs(int top) const {
    if (top <= 0 || logs.empty()) {
        return {};
    }

    size_t count = min(static_cast<size_t>(top), logs.size());
    return vector<LogTransaksiEntry>(logs.end() - count, logs.end());
}
```

Pada kelas LogTransaksiGame, digunakan juga komponen dari C++ Standard Library yaitu STL Algorithm, khususnya fungsi std::min. Fungsi ini digunakan pada method getLogs(int top) untuk menentukan jumlah maksimum elemen log yang akan diambil tanpa melebihi ukuran vector yang tersedia. Dengan menggunakan std::min, program dapat secara aman menangani kasus ketika nilai top lebih besar dari jumlah log yang ada, sehingga menghindari akses indeks yang tidak valid. Penggunaan STL Algorithm ini membuat kode lebih ringkas, aman, dan mudah dipahami dibandingkan jika harus melakukan pengecekan manual menggunakan kondisi if-else.

Selain itu, dalam implementasi sistem, digunakan beberapa komponen dari C++ STL lainnya seperti std::string untuk manipulasi teks yang fleksibel dan memudahkan pengelolaan data berupa nama atau kode, std::pair untuk menyimpan dua nilai yang saling berhubungan secara sederhana tanpa perlu membuat struktur tambahan, serta std::exception untuk *base class* NimonspoliException untuk sistem penanganan error yang terstruktur dan konsisten, dan std::mt19937 dari library <random> untuk kebutuhan pengacakan dadu karena menyediakan generator bilangan acak yang baik dan stabil. Yang mana kode dari masing-masing penggunaan STL telah ditunjukkan pada bagian sebelumnya.

4.6. Konsep OOP lain

Seperti dijelaskan di poin-poin sebelumnya, permainan ini menggunakan **abstract base class** pada bagian kartu yakni class Kartu dan class KartuKemampuanSpesial. Kedua class tersebut dikategorikan sebagai abstract base class karena merupakan dasar dari class-class yang lain dan kedua kelas tersebut bersifat abstrak atau tidak bisa diinisiasi. Kedua kelas tersebut juga memiliki **pure virtual function**.

Pada sistem ini, konsep **composition** digunakan pada beberapa bagian, salah satunya pada hubungan antara kelas Papan dengan Petak serta ConfigData dengan PropertiConfig dan AksiConfig. Pada kelas Papan, hal ini terlihat dari atribut vector<Petak*> daftarPetak, di mana objek Petak dibuat secara dinamis di dalam konstruktor menggunakan new dan dihapus pada destructor menggunakan delete, sehingga lifecycle Petak sepenuhnya bergantung pada Papan. Selain itu, pada kelas ConfigData, relasi composition ditunjukkan melalui penyimpanan objek PropertiConfig dan AksiConfig secara langsung dalam map, yang di-move ke dalam objek ConfigData saat inisialisasi. Dengan demikian, objek-objek tersebut tidak berdiri sendiri dan sepenuhnya dikelola oleh ConfigData. Hal ini memastikan bahwa setiap komponen hanya hidup selama objek induknya ada, sehingga struktur program menjadi lebih terkontrol dan konsisten.

Relasi **aggregation** dalam sistem ini terdapat pada kelas PlayerActionService, yang menyimpan referensi ke berbagai komponen seperti ManagerProperti, ManagerPenjara, ManagerFestival, ConfigData, dan daftar pemain dalam bentuk pointer. Objek-objek tersebut tidak dibuat maupun dihancurkan oleh PlayerActionService, melainkan disediakan dari luar dan hanya digunakan selama eksekusi program. Lifecycle objek-objek tersebut tidak bergantung pada PlayerActionService, sehingga relasi ini termasuk aggregation, di mana suatu kelas menggunakan objek lain tanpa memiliki kontrol penuh terhadap keberadaannya. Digunakan aggregation sebab PlayerActionService memiliki peran sebagai pengelola aksi pemain yang membutuhkan akses ke berbagai komponen sistem tanpa harus memiliki atau mengatur lifecycle-nya, sehingga meningkatkan fleksibilitas dan memisahkan tanggung jawab antar kelas (SRP).

5. Bonus Yang dikerjakan

5.1. Bonus Kreasi Mandiri

Bonus kreasi mandiri yang dibuat yakni membuat KartuKemampuanSpesial tambahan. Tiga kartu kemampuan baru ditambahkan sebagai tambahan dari KartuKemampuanSpesial yang sudah ada. Tiga kartu tersebut terdiri dari RotasiKartuCard, ReverseCard, dan PenjaraKanCard. RotasiKartuCard membuat seluruh kartu di tangan setiap pemain akan berputar kepemilikan searah giliran bermain. Pemain A akan mendapat kartu pemain B, pemain B akan mendapat kartu pemain C, dan seterusnya. ReverseCard akan membalik arah giliran bermain. Sementara itu, PenjaraKanCard membuat seorang pemain dapat mengirim salah satu lawan untuk masuk ke penjara.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Konfigurasi Data	13524042	13524042
Transaction Logger	13524042	13524042
Pemain, giliran, dan dadu	13524042	13524042
Exception	13524042	13524042
Save/Load	13524042	13524042
Parsing Config File	13524042	13524042

PlayerActionService	13524091	13524091
Game UI & rendering papan	13524091	13524091
Command parser & dispatcher	13524091	13524091
Main program & inisialisasi	13524091	13524091
Makefile	13524091	13524091
Properti	13524073	13524073
Manager Properti	13524073	13524073
Petak	13524073	13524073
Papan	13524073	13524073
Petak Properti	13524073	13524073
Deck Kartu	13524109	13524109
Kartu	13524109	13524109

Kartu Kemampuan Spesial	13524109	13524109
Bonus Tambahan Kartu	13524109	13524109
Petak Kartu	13524109	13524109
Petak Festival dan Manager Festival	13524109	13524109
Movement Controller Pemain dan Game	13524087	13524087
Manager Penjara	13524087	13524087
Manager Giliran dan Game Engine	13524087	13524087
Petak Aksi Spesial	13524087	13524087
Kebangkrutan	13524087	13524087
Endgame	13524087	13524087
Formatter	13524087	13524087

Lampiran

Lampiran A. Form Asistensi

IF2010_TB1_Asistensi_GTW.docx.pdf